

Campus Eats Process Document

1. INTRODUCTION.....	2
1.1. Purpose of the System.....	2
1.2 Problem Statement.....	2
1.3 Proposed Solution.....	3
2. ORGANIZATIONAL CONTEXT.....	4
2.1 Organization Overview.....	4
2.2 Business Objectives.....	4
3. STAKEHOLDER IDENTIFICATION.....	5
3.1 Primary Stakeholders.....	5
3.2. Secondary Stakeholders.....	5
4. SYSTEM SCOPE AND BOUNDARIES.....	6
4.1 In Scope.....	6
5. BUSINESS PROCESS OVERVIEW.....	7
5.1 Order Placement Process.....	7
5.2 Vendor Order Management Process.....	7
6. FUNCTIONAL REQUIREMENTS.....	8
6.1. Student Functional Requirements.....	8
6.2 Vendor Functional Requirements.....	8
6.3 Administrator Functional Requirements.....	8
7. NON-FUNCTIONAL REQUIREMENTS.....	9
8. SYSTEM ARCHITECTURE.....	10
8.1 High-Level Architecture.....	10
8.2 Core System Modules.....	10
9. DATABASE DESIGN OVERVIEW.....	11
9.1 Core Entities.....	11
9.2 Key Relationships.....	11
10. USER INTERFACE DESIGN CONSIDERATIONS.....	12
10.1 Student Interface.....	12
10.2 Vendor Interface.....	12
10.3 Administrator Interface.....	12
11. FEASIBILITY ANALYSIS.....	13
11.1 Technical Feasibility.....	13
11.2 Economic Feasibility.....	13
11.3 Operational Feasibility.....	13
12. CONCLUSION.....	14

1. INTRODUCTION

1.1. Purpose of the System

Campus Eats is a platform for campus food ordering and pick-up management systems designed for Rosebank International University College. The system enables customers to order food from registered campus vendors, allows vendors to manage menus and incoming orders, permits delivery personnel to handle food deliveries, and provides administrators with full system management capabilities.

The purpose of the system is to digitize the campus food ordering process, improve operational efficiency, reduce queue times, and provide real-time order tracking and reporting.

1.2 Problem Statement

In many campus environments, food ordering is conducted physically at vendor stalls. This creates several operational challenges:

1. Long queues during peak hours.
2. Order miscommunication between students and vendors.
3. No structured digital order tracking.
4. Manual payment handling.
5. Lack of transaction records.
6. Inefficient coordination between vendors and pick-up personnel.
7. There is currently no integrated system that centralizes ordering, payment processing, vendor management, and delivery coordination within the campus environment.

1.3 Proposed Solution

Campus Eats provides:

1. A customer interface for browsing menus and placing orders.
2. A vendor portal for managing menu items and processing orders.
3. An interface for managing pick-ups.
4. An administrative dashboard for full system control and reporting.

The system is designed to operate within the boundaries of a single campus.

2. ORGANIZATIONAL CONTEXT

2.1 Organization Overview

The system is intended for implementation within a higher education institution that contains:

1. Multiple campus food vendors.
2. Registered customers.
3. Administrative staff.
4. The platform supports structured digital interaction between these parties.

2.2 Business Objectives

The organization aims to:

- Improve take away service for customers.
- Increase vendor operational efficiency.
- Improve revenue tracking and reporting.
- Reduce physical congestion at vendor stalls.
- Provide management with structured data insights.

3. STAKEHOLDER IDENTIFICATION

3.1 Primary Stakeholders

1. Students
 - a. Customers who browse menus, place orders, and make payments.
2. Standard
 - a. Customers who browse menus, place orders, and make payments.
3. Vendors
 - a. Campus food stalls and cafeterias that prepare and fulfil orders.
4. System Administrator
 - a. Staff responsible for managing users, vendors, reports, and system configuration.

3.2 Secondary Stakeholders

1. Campus management
2. IT support department
3. Finance department

4. SYSTEM SCOPE AND BOUNDARIES

4.1 In Scope

1. User registration and authentication.
2. Role-based access control.
3. Menu browsing and search functionality.
4. Cart management.
5. Order placement and confirmation.
6. Order status tracking.
7. Administrative reporting.

The system is limited to one campus environment.

5. BUSINESS PROCESS OVERVIEW

5.1 Order Placement Process

1. Customers log into the system.
2. Customers browse vendor menus.
3. Customers select items and add them to the cart.
4. Customers confirm orders and make payment online (Debit Card, Campus Eats Wallet).
5. Vendors receive order notification.
6. Vendors accept or reject the order.
7. Vendors prepare the order.
8. The order is ready to be picked up by the student.
9. Order status is updated to “ready for pick up”.

5.2 Vendor Order Management Process

1. Vendor logs into the system.
2. Vendor views pending orders.
3. Vendor accepts or rejects the order.
4. Vendor updates preparation status.
5. Vendor marks order as ready for pick-up.

6. FUNCTIONAL REQUIREMENTS

6.1. Customer Functional Requirements

The system shall allow students to:

1. Register and log in.
2. Browse and search vendor menus.
3. Add items to cart.
4. Modify cart contents.
5. Place orders.
6. Track order status.
7. View order history.

6.2 Vendor Functional Requirements

The system shall allow vendors to:

1. Register vendor profiles.
2. Add new items on the menu.
3. Update existing menu items.
4. Remove menu items.
5. Set item availability.
6. Accept or reject orders.
7. Update order status.

6.3 Administrator Functional Requirements

The system shall allow administrators to:

1. Manage user accounts.
2. Manage vendor accounts.
3. View system reports.
4. Monitor transactions.
5. Deactivate or suspend accounts.

7. NON-FUNCTIONAL REQUIREMENTS

1. The system must support at least 1000 concurrent users.
2. System response time must not exceed 3 seconds per request.
3. The system must implement secure authentication mechanisms.
4. Data must be stored securely in a relational database.
5. The system must be accessible via modern web browsers.
6. The system must ensure data integrity and confidentiality.

8. SYSTEM ARCHITECTURE

8.1 High-Level Architecture

The system follows a three-tier architecture:

1. Presentation Layer.
 - a. Platform interfaces are accessed by customers, vendors, and administrators.
2. Application Layer.
 - a. Backend server that handles business logic, authentication, order processing, and reporting.
3. Data Layer.
 - a. Relational database server that stores persistent system data.

8.2 Core System Modules

1. Authentication and Authorization Module.
2. User Management Module.
3. Vendor Management Module.
4. Menu Management Module.
5. Order Processing Module.
6. Payment Management Module.
7. Pick-up Management Module.
8. Reporting Module.

Each module interacts with the central database through structured queries.

9. DATABASE DESIGN OVERVIEW

9.1 Core Entities

1. Users.
2. Roles.
3. Vendors
4. Menu_Items
5. Orders
6. Order_Items
7. Payments

9.2 Key Relationships

1. A User is assigned to one Role.
2. A Vendor has many Menu_Items.
3. A Student places many Orders.
4. An Order contains many Order_Items.
5. An Order has one Payment record.

The database will be normalized to Third Normal Form to ensure minimal redundancy and strong data integrity.

10. USER INTERFACE DESIGN CONSIDERATIONS

10.1 Customer Interface

1. “Home” dashboard that displays all vendors and available items.
2. “My Order” dashboard for receipts.
3. “Cart” dashboard that displays all items that the user wishes to purchase.
 - a. Should have a service fee.
 - i. Under 500 rand, 10% service fee.
 - ii. Between 500 and 1,000 rand, 5% service fee.
 - iii. 1,000 rand and above, 2.5% service fee.
 - b. Tax of 20%.
 - c. Round amount to the next 5 rand.
 - d. Total.
 - e. Students get 2.5% discounts.
4. “Feedback” dashboard to submit a compliment or complaint. It should have a selection between a complaint and compliments, subject (maximum 200 characters), main message (maximum 2000 characters).

10.2 Vendor Interface

1. “Home” dashboard that displays the vendors’ summary. The summary would contain a number of menu items, active orders, overall revenue, shop status, incoming orders, and recent complete orders.
2. “Menu” dashboard for the vendors to create, replace, update, and delete item information.
3. “Orders” dashboard that is responsible for displaying all orders, pending orders, active orders, and complete orders.
4. “Reports” dashboard that is responsible for displaying all orders for the past week, current month, current year.

10.3 Administrator Interface

1. “Home” dashboard that displays the total users, active vendors, total menu items from all vendors, total orders, pending approvals, recent orders, revenue by vendor, order status distribution.
2. “User Management” dashboard that displays all users (categorized by standard, students, vendors, admins, and pending), create new users, filter users category (All Users, Pending, Active, Standard, Vendors, Admins), and display all users and their information (User ID, Name, Username, Email, Role, Actions - Suspend and Delete).
3. “Vendor Management” dashboard that displays all vendors and status. There are filters for all vendors, pending approval, approved, suspended.

4. “Order Management” dashboard that displays all orders and copies of all receipts. All receipts should have the order number, customer, vendor, total, status, timestamps, actions. There should be filters for all, pending, accepted, preparing, ready, completed, cancelled.
5. “Transactions” dashboard that displays only the total transactions, total revenue, pending payments, average transactions.
6. “Reports” dashboard displays the total transactions, total revenue, revenue order value, unique customers, and active vendors.
7. “Feedback” dashboard that displays all complaints and compliments. Filter by compliment, complaint, and all. Filter the compliment and complaint by resolved and unresolved.

All interfaces will maintain consistency, logical navigation flow, and usability standards.

11. FEASIBILITY ANALYSIS

11.1 Technical Feasibility

The system can be developed using standard web development technologies, including:

1. Frontend framework.
2. Backend server framework.
3. Relational database management system.

No specialized hardware infrastructure is required.

11.2 Economic Feasibility

Development costs are limited to software development time and infrastructure hosting.

The system improves operational efficiency and provides financial transparency.

11.3 Operational Feasibility

Customers and vendors are familiar with web-based ordering systems. Minimal training will be required.

12. CONCLUSION

Campus Eats is a structured, scalable, and feasible campus food ordering and delivery management system. The system demonstrates:

1. Clearly defined stakeholders.
2. Structured business processes.
3. Comprehensive functional requirements.
4. Strong database modelling opportunities.
5. Logical system architecture.
6. GUI design capability.
7. Opportunities for use case and system modelling.

The system is realistic, technically achievable within a semester timeframe, and academically suitable for demonstrating software engineering analysis and design competencies.